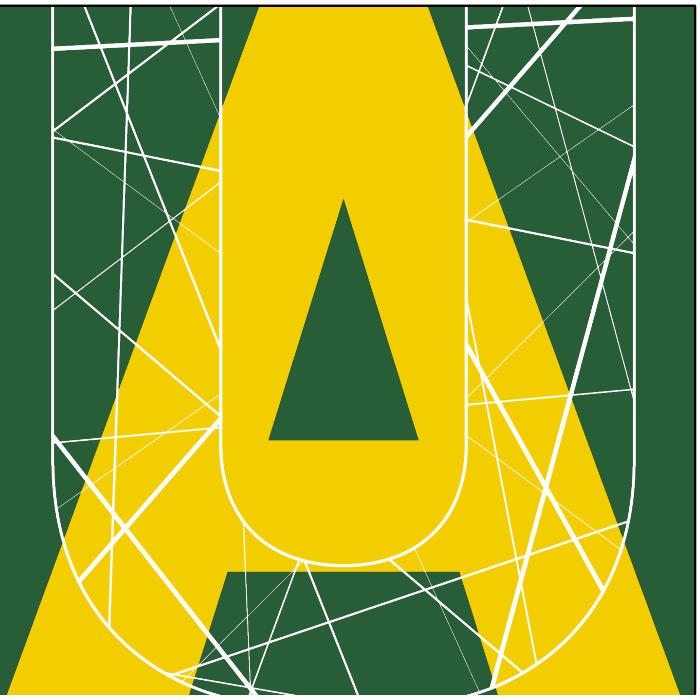
ECE 315

Steven Knudsen, PhD, PEng

Lecture 19:

- Parallel Interfacing
- Busses

UNIVERSITY OF ALBERTA



Last Lecture

- Inter-Integrated Circuit (I²C)

UNIVERSITY OF ALBERTA

This Lecture

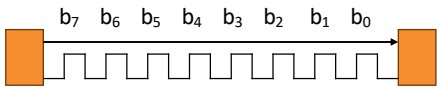
- Parallel Interfacing
- Busses

Learning Objectives

- Distinguish Between Serial and Parallel Communication
  - Explain the difference between serial and parallel
  - Identify common serial (UART, SPI, I²C) and parallel (PCI, SCSI, IDE/ATA, ISA) protocols
- Understand the Structure and Purpose of Computer Busses
  - Describe what a bus is and how it functions as a shared communication pathway
  - Explain the role of the data, address, and control buses
- Interpret Timing Diagrams for Synchronous and Asynchronous Parallel Buses
  - Read and explain synchronous bus timing (clocked transfers).
  - Understand asynchronous handshaking using REQ/ACK signals.
- Evaluate the Pros and Cons of Parallel Busses
- Summarize When and Why Parallel Busses Are Used

Serial vs Parallel

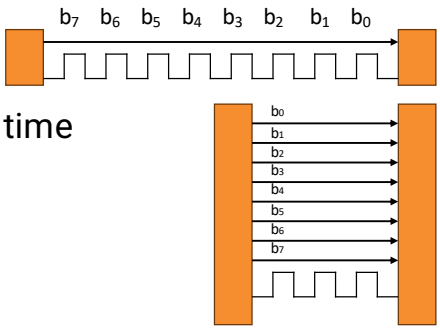
- A serial bus sends bits one at a time over a single channel



UNIVERSITY OF ALBERTA

Serial vs Parallel

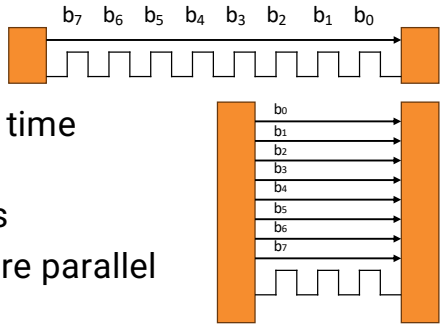
- A serial bus sends bits one at a time over a single channel
- A parallel bus sends multiple bits at a time over multiple channels



UNIVERSITY OF ALBERTA

Serial vs Parallel

- A serial bus sends bits one at a time over a single channel
- A parallel bus sends multiple bits at a time over multiple channels
- UART, SPI, and I^2C are serial protocols
- PCI, SCSI, IDE or ATA, ISA, and GPIB are parallel protocols



- Parallel was once common for commercial and household, not now

UNIVERSITY OF ALBERTA

Parallel Busses

- Used in embedded, legacy, and high-speed applications, parallel busses offer
  - A time-proven design approach
  - Simple protocols
  - Potentially high data-rate transfers
  - Low latency
  - Easy expansion (more bits in parallel)

Parallel vs Serial

Serial Transmission

- One bit at a time
- Slower per clock cycle
- Long distance, especially wireless protocols
- Compact, only a few wires
- Inexpensive
- Example, USB, Ethernet, SPI

Parallel Transmission

- Many bits at a time
- Faster per clock cycle
- Short distance; e.g. across boards, inside laptops, phones
- Many channels take physical space
- Expensive
- Example, PCIe inside a PC

https://eduinput.com/wp-content/uploads/2025/02/Difference-Between-Serial-Transmission-and-Parallel-Transmission-1024x576.webp

Bus

- A **bus** is a shared communication pathway that connects multiple devices
- Used to connect components in a computer
- Used to connect computers, but only over relatively short distances
  - Less and less these days except for high-performance systems
  - More common to connect local computers using Ethernet or WiFi

Wikipedia contributors. "Bus (computing)." *Wikipedia, The Free Encyclopedia*. Wikipedia, The Free Encyclopedia, 15 Nov. 2025. Web. 20 Nov. 2025.

System Bus

- Within a computer system, the System Bus connects the CPU with supporting peripherals,
  - input/output (mouse and keyboard controllers,
  - volatile memory (RAM),
  - non-volatile (hard or solid-state disks),
  - sound cards, etc.
- Typically, the CPU is the bus controller; all other devices are peripherals
- In a computer, a CPU is connected to peripherals via the **Data**, **Address**, and **Control** busses

Wikipedia contributors. "Bus (computing)." *Wikipedia, The Free Encyclopedia*. Wikipedia, The Free Encyclopedia, 15 Nov. 2025. Web. 20 Nov. 2025.

Computer Busses

Data

- Carries binary data between components
- Width determines data transfer size (8, 16, 32 bits, etc.)
- Wider bus → higher bandwidth

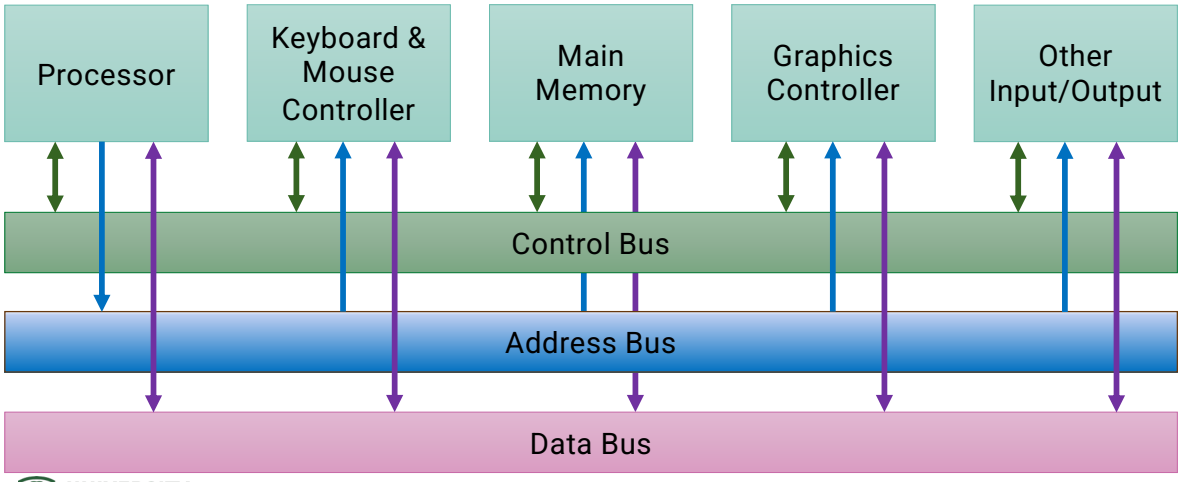
Address

- Specifies memory or I/O device location
- Unidirectional from CPU to peripherals
- Width determines max addressable space

Control

- Manages coordination and timing
- Read/Write signals
- Interrupt request
- Clock signals
- Bus enable and strobe signals

Single Bus System



UNIVERSITY OF ALBERTA

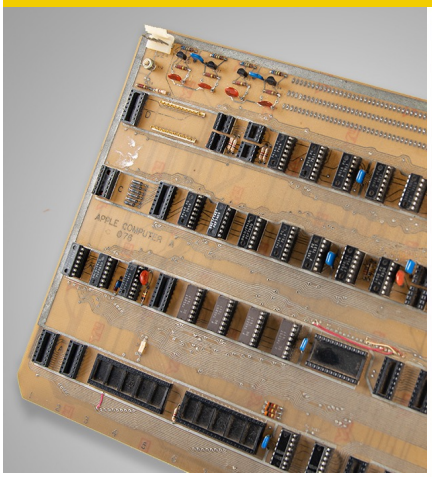
Wikipedia contributors. "Bus (computing)." *Wikipedia, The Free Encyclopedia*. Wikipedia, The Free Encyclopedia, 15 Nov. 2025. Web. 20 Nov. 2025.

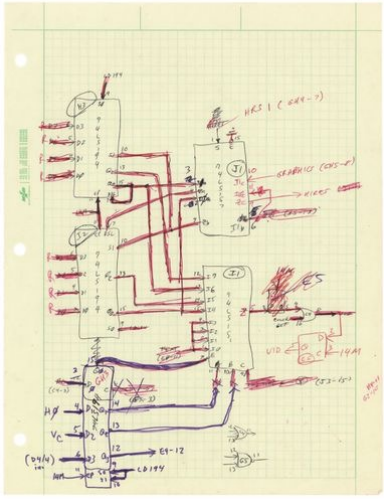
13

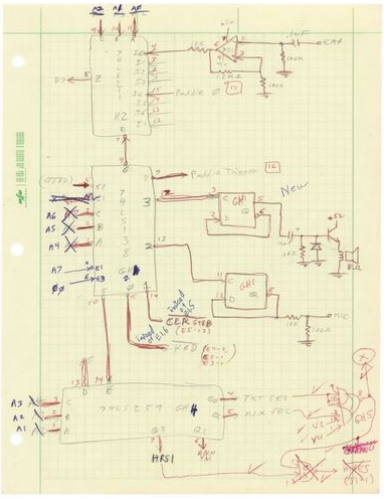
Expansion / Peripheral Busses

- An expansion bus extends the system to connect additional peripherals
- Rely on a standard definitions to enable third-party devices
- For example,
  - The Peripheral Component Interconnect Express (PCIe) high-speed standard connects internal peripherals to the CPU
    - Most often graphics cards, but can also be DVD/BluRay read/writers, FPGA-based accelerators, solid-state disks, RAID and storage controllers, or fast interconnect devices like fiber-channel host adapters, fast Ethernet network interface card/controllers (NICs)
  - Universal Serial Bus (USB) connects to external peripherals
    - Removable storage (Direct Attached Storage, DAS)
  - High-Definition Multimedia Interface (HDMI)
    - Attach to game systems, audio-visual systems

Apple I & II





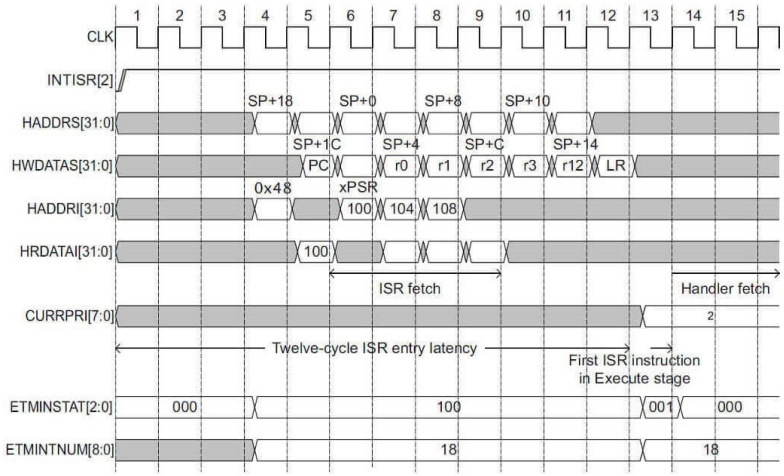


The original Apple I board showing parallel interfaces

Wozniak's sketches for the Apple II with more parallel channels.

Parallel Bus Signaling – ARM example

- Example ARM Cortex-M3 computer bus timing
- Shows signaling that happens for interrupts
- 2, 32-bit address busses
- 2, 32-bit data busses
- CURRPRI[7:0], 8-bit Current interrupt priority level
- ETMINSTAT[2:0], 3-bit debug interrupt trace status
- ETMINTNUM[8:0], 9-bit interrupt number



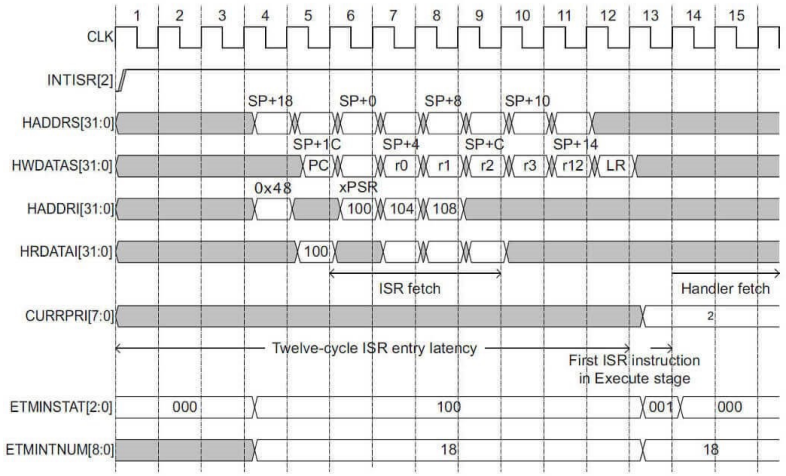
https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/beginner-guide-on-interrupt-latency-and-interrupt-latency-of-the-arm-cortex-m-processors

This timing diagram illustrates for a processor a lot of parallel data being exchanged every clock pulse

The example has to do with interrupt handling, and we can see that by moving a lot of data quickly, we can be very responses, keep the latency low.

Parallel Bus Signaling – ARM example

- Example of semi-synchronous operation
- The exchange is initiated by the INTISR signal going high
- The signals show what is set up prior to the ISR instructions executing

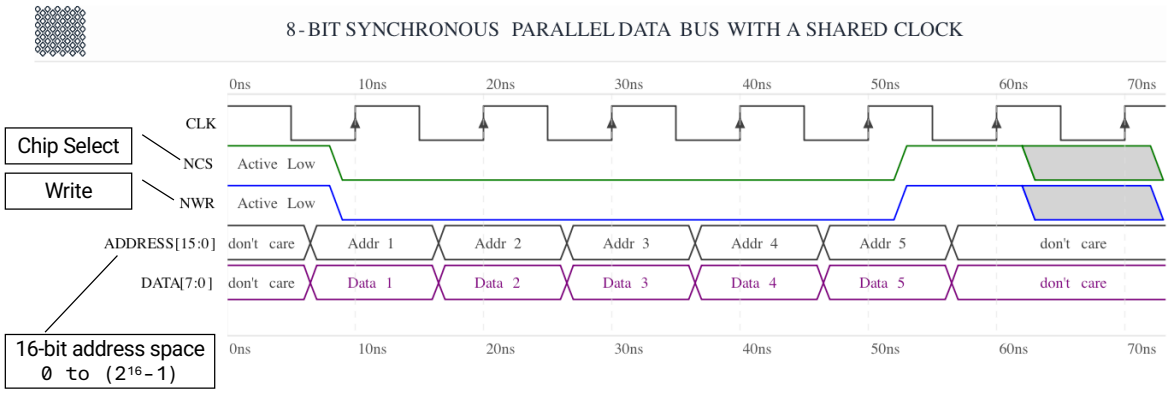


https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/beginner-guide-on-interrupt-latency-and-interrupt-latency-of-the-arm-cortex-m-processors

This timing diagram illustrates for a processor a lot of parallel data being exchanged every clock pulse

The example has to do with interrupt handling, and we can see that by moving a lot of data quickly, we can be very responses, keep the latency low.

8-bit synchronous parallel data bus w/ shared clock



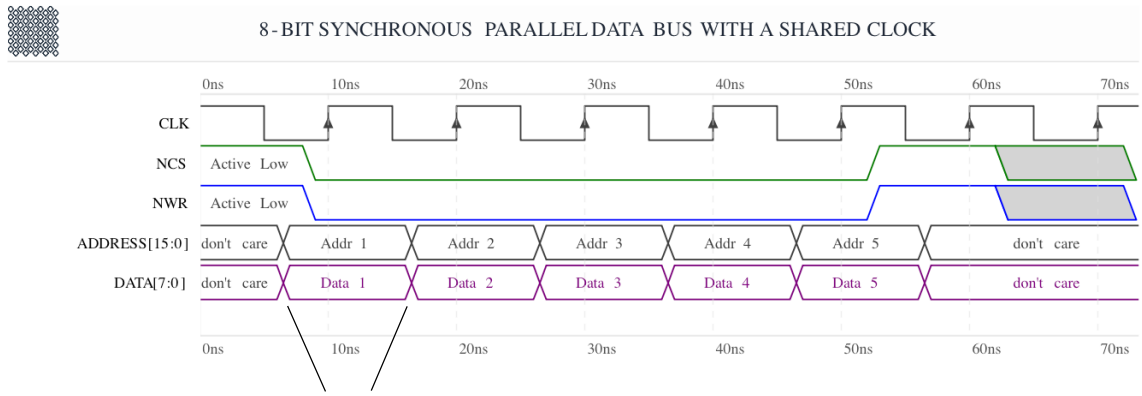
UNIVERSITY OF ALBERTA

Made with help from ChatGPT

Made with Tiagram www.tiagram.com

18

8-bit synchronous parallel data bus w/ shared clock



Signals stable before clock edges

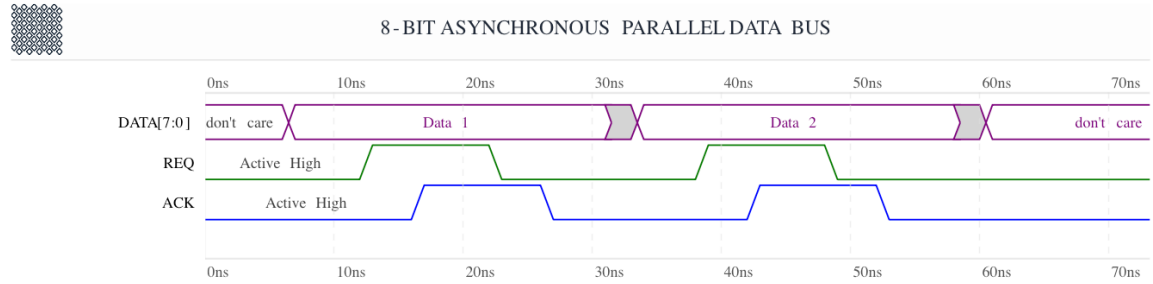
UNIVERSITY OF ALBERTA

Made with help from ChatGPT

Made with Tiagram www.tiagram.com

19

Simple 8-bit asynchronous parallel data bus



1. Data 1 becomes valid on bus (sender holds it steady).
2. Sender sets REQ high.
3. Receiver sees REQ = 1, latches Data 1 , then raises ACK.
4. Sender sees ACK = 1, then drops REQ low.
5. Receiver sees REQ return to 0, then drops ACK low.
6. → Handshake for Data 1 is complete.

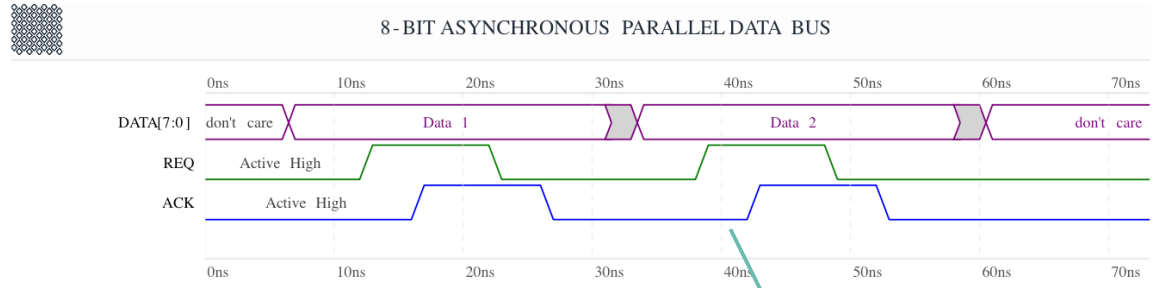
UNIVERSITY OF ALBERTA

Made with help from ChatGPT

20

This sequence repeats each time the sender has data to provide to the receiver.

Simple 8-bit asynchronous parallel data bus



1. Data 1 becomes valid on bus (sender holds it steady).
2. Sender sets REQ high.
3. Receiver sees REQ = 1, latches Data 1 , then raises ACK.
4. Sender sees ACK = 1, then drops REQ low.
5. Receiver sees REQ return to 0, then drops ACK low.
6. → Handshake for Data 1 is complete.

Handshaking Signals

UNIVERSITY OF ALBERTA

Made with help from ChatGPT

21

This sequence repeats each time the sender has data to provide to the receiver.

Parallel Bus Standards - ARM

AXI (AMBA AXI4, AXI4-Lite, AXI4-Stream)
(Advanced eXtensible Interface)

- Industry-standard on-chip bus for nearly all modern SoCs
- Highly parallel: multiple address, data, and control channels
- Used in CPUs, GPUs, FPGAs (Xilinx/AMD, Intel), ASICs
- AXI4 full = high-performance memory-mapped
- AXI4-Lite = control registers
- AXI4-Stream = parallel streaming data (no addressing)

AHB (AMBA AHB-Lite / AHB5)
(Advanced High-performance Bus)

- Still common in microcontrollers
- Parallel address/data bus
- Used in ARM Cortex-M devices (e.g., ST, NXP, Microchip SAM, TI)
- Lower complexity than AXI, remains widely used today

Parallel Bus Standards - PCs

PCIe with Parallel PHY Lanes (Peripheral Component Interconnect Express)

- Not a classical parallel bus at the board level, but internally parallel in lane groups.
- Modern PCIe (4.0/5.0/6.0) uses multiple parallel differential lanes
- PHY is ***serialized*** per lane, but multiple lanes operate in parallel
- Used in nearly every contemporary computer system
- Included here because modern hardware engineers treat the multi-lane link as a "parallel bus" at the logical level

DDR (Double Data Rate) Memory Interfaces (DDR3, DDR4, DDR5 LPDDR4/5)

- Still fundamentally parallel data buses, despite high speed
- 8-, 16-, 32-bit wide DQ buses with strobes (DQS)
- Extremely contemporary—used in all PCs, smartphones, servers

Parallel Bus Standards - Embedded

QSPI / Octal SPI (OSPI) (Queued Serial Peripheral Interface)

- Not fully “parallel” in the traditional sense, but multi-bit-wide synchronous transfers (4- or 8-bit)
- Used for modern NOR flashes and execute-in-place (XiP) systems
- Supported in many microcontrollers in 2024/2025

eMMC Parallel Interface (embedded MultiMediaCard)

- NAND flash interface used in smartphones, embedded boards
- 8-bit parallel data bus
- Still widely produced and supported (v5.1 current)

Parallel Bus/Interface Pros and Cons

Pros

- High throughput
- Low complexity
- Easy to interface with microcontrollers
- Good for short-distance communication

Cons

- More wires → bulkier connectors
- Susceptible to noise over long distances
- Lower scalability than serial buses
- Replaced by serial (USB, PCIe) in modern systems

Summary

- Parallel busses support high-data rate transfers
- Common for processor architecture and internal computer transfer
- Simple timing or handshaking requirements
- Take up space due to many wires
- Not great for interfacing between physical systems
  - Connectors are bulky, hard to connect/disconnect, expensive
- Many serial protocols are really fast, so parallel not as needed
- Still the way to go for very high-performance systems

Next Lecture

- Computer Interfaces, in general
- Planning peripherals and interfaces
- Design and build for test
- RTM

Questions

UNIVERSITY OF ALBERTA